

Programmation Orientée objet Java

Pr. Ilyass OUAZZANI TAYBI

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence SI S5

Année universitaire : 2025/2026

Exceptions en Java

Qu'est-ce qu'une exception ?

- Une **exception** est un **événement anormal** qui se produit pendant l'exécution d'un programme et qui **interrompt le flux normal des instructions**. Elle signale qu'une erreur ou une situation inattendue est survenue et peut être gérée par le programme.
- Définies par des mots clés du langage
 - **try, catch, finally, throw et throws**
- Une **exception** est un **objet**
- Une **classe d'exception** pour chaque sorte d'erreur

Exceptions en Java

Propagation des Exceptions

Lors de la détection d'une exception, un **objet** qui hérite de la classe **Exception** est créé (on dit qu'une exception est levée) et propagé à travers la pile d'exécution jusqu'à ce qu'il soit traité.

Exemple : Exception non capturée

```
public class ExceptionExample {  
    public static char charAt(char[] array,int index)  
    {  
        return array[index];  
    }  
  
    public static void main(String[] args) {  
        char[] array = args[0].toCharArray();  
        charAt(array,0);  
    }  
}
```

```
@ilyasss-MacBook-Pro-2 Ilyass % java ExceptionExample
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for  
length 0
```

```
    at ExceptionExample.main(ExceptionExample.java:6)
```

Exceptions en Java

Quand surviennent les exceptions ?

Quatre cas possibles d'exception en Java :

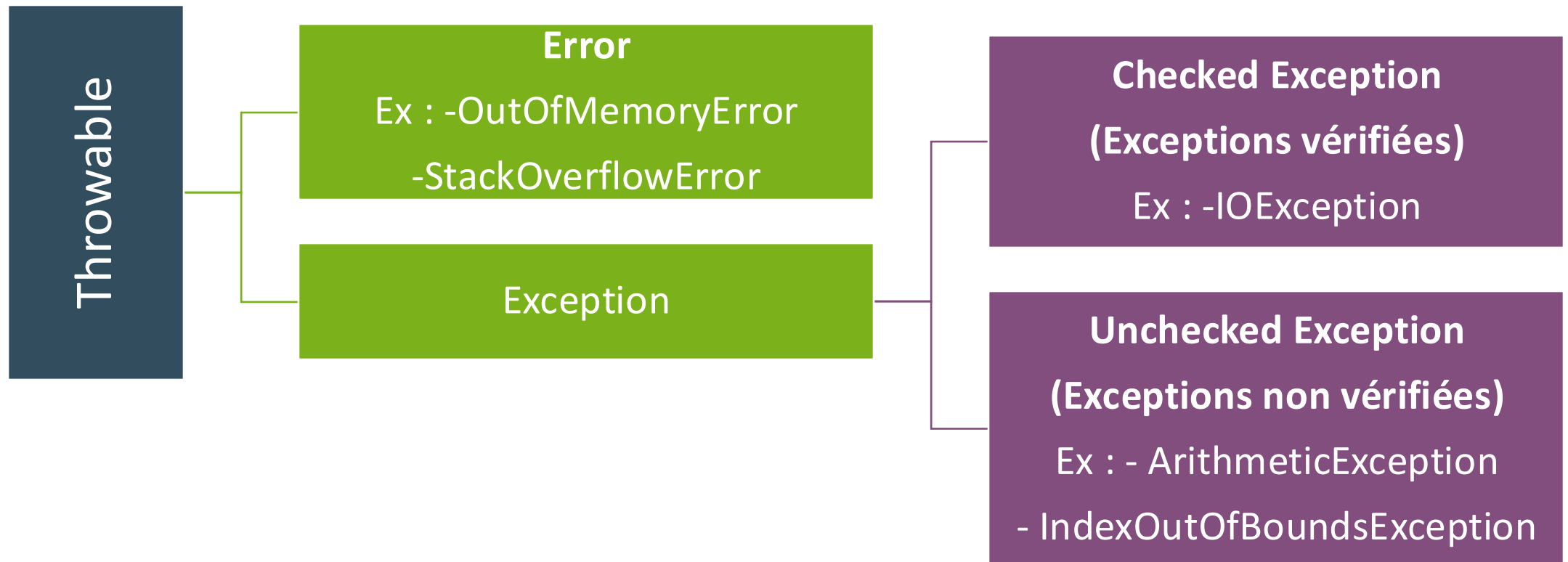
- 1. Erreur interne de Java** : Problèmes graves du système Java lui-même, souvent non récupérables. Exemple : *OutOfMemoryError*, *StackOverflowError*.
- 2. Erreur de programmation ou de déroulement** : Problèmes qui apparaissent à cause du code ou des conditions d'exécution. Exemples : *ArrayIndexOutOfBoundsException*, *IOException*, *SocketException*
- 3. Appel d'une méthode qui lève une exception** : Certaines méthodes peuvent générer des exceptions qu'on doit gérer. Exemple : *FileReader* peut lancer *FileNotFoundException*.
- 4. Lancement volontaire d'une exception avec throw** : Le programmeur peut créer et lancer une exception pour signaler une condition particulière. Exemple :

```
if (age < 18) {  
    throw new IllegalArgumentException("Age doit  
être ≥ 18");  
}
```

Exceptions en Java

Types d'exceptions

Il existe 3 trois types d'exceptions organisés comme ceci :



Exceptions en Java

Types d'exceptions

- Les **Error** représentent des **erreurs graves liées au fonctionnement interne de la JVM**. Il est **très rare (et déconseillé) de les attraper**, car elles correspondent à des situations dont le programme ne peut généralement pas se remettre. Exemples : `OutOfMemoryError`, `StackOverflowError`.
- Les **exceptions non vérifiées (`RuntimeException`)** sont des **exceptions que l'on peut attraper**, mais que **l'on n'est pas obligé de gérer**. Elles correspondent souvent à des erreurs de programmation. Elles ne sont pas vérifiées à la compilation. Exemples : `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NullPointerException`.
- Les **exceptions vérifiées**, qui héritent de **Exception sans passer par `RuntimeException`**, doivent être traitées soit avec un bloc **try/catch**, soit déclarées avec **throws**. Elles correspondent à des **situations prévisibles** comme les erreurs d'entrée/sortie ou de base de données, par exemple `IOException` et `SQLException`.

Exceptions en Java

Le mot-clé try

- Un bloc try contient des instructions susceptibles de lever une exception :

```
try {  
    // instructions à risque  
}
```

- Les instructions du bloc try s'exécutent jusqu'à ce qu'elles se terminent normalement ou qu'une exception soit levée, interrompant alors l'exécution normale du bloc.

Exceptions en Java

Le mot-clé catch

- Le bloc **catch** permet d'intercepter et de traiter une exception :

```
catch (UneException e) {  
    // traitement de l'exception  
}
```

- Lorsqu'une exception est attrapée, on peut soit remettre le programme dans un état stable et poursuivre l'exécution, soit effectuer un nettoyage (libérer des ressources, fermer des fichiers, etc.) puis relancer l'exception.
- La pire pratique consiste à écrire :

```
catch (Exception e) { }
```

- Car cela **masque les erreurs** sans les traiter, rendant le débogage difficile.

Exceptions en Java

Les règles du catch

- On peut placer plusieurs blocs catch à la suite d'un même bloc try, à condition que chaque catch traite un type d'exception différent :

```
try {  
    // instructions à risque  
} catch (ExceptionA e) {  
    // traitement spécifique à ExceptionA  
} catch (ExceptionB e) {  
    // traitement spécifique à ExceptionB  
}
```

- Lorsqu'une exception survient, un seul bloc catch est exécuté, celui qui correspond au type de l'exception levée (ou à une de ses superclasses).

Exceptions en Java

Les règles du catch

- L'ordre des blocs **catch** est important : le premier bloc correspondant au type de l'exception sera exécuté.
- Il faut donc placer les types d'exceptions les plus spécifiques en premier et les plus génériques (comme `Exception`) en dernier, afin d'éviter qu'elles ne masquent les autres.

```
try { // instructions à risque
} catch (ExceptionA e) { // traitement spécifique à ExceptionA
} catch (ExceptionB e) { // traitement spécifique à ExceptionB
}
```

Exceptions en Java

Le mot-clé finally

- Sert à exécuter un code quoi qu'il arrive (fermer un fichier, une connexion, libérer une ressource)

```
public class SimpleFinallyExample {  
    public static void main(String[] args) {  
        try {  
            System.out.println("Début du try");  
            int x = 5 / 0; // Lance une ArithmeticException  
        } finally { System.out.println("Bloc finally exécuté"); }  
        System.out.println("Fin du programme");  
    }  
}
```

- **Remarque :** Un bloc try ne peut jamais être utilisé seul : il doit toujours être suivi d'au moins un bloc catch, ou d'un bloc finally, ou des deux.

Exceptions en Java

Le mot-clé throws

- Le mot-clé **throws** permet de **déclarer précisément les exceptions qu'une méthode peut générer**.
- Il est utilisé pour toutes les exceptions que la méthode ne peut pas gérer elle-même.
- Lors de sa déclaration, il est conseillé de spécifier les exceptions les plus spécifiques possible, afin de donner à l'appelant une information claire sur les erreurs à anticiper.

Exceptions en Java

Le mot-clé throw

- En Java, tout événement pouvant être levé est un objet héritant de **java.lang.Throwable**. Les **exceptions gérables** héritent de la classe **Exception**, tandis que les **erreurs liées à la JVM** héritent de la classe **Error**.
- L'instruction **throw** permet de **lever explicitement une nouvelle exception**, en créant une instance de cette classe :

```
throw new Exception("Message d'erreur");
```

- C'est la seule manière primitive de générer une exception dans un programme, les autres exceptions étant généralement levées automatiquement par la JVM lors d'erreurs d'exécution (comme une division par zéro ou un accès à un indice hors limites).

Exceptions en Java

Un exemple de throws

```
public int compte(int no) throws CompteInvalideException
{
    if (no != 0)
        return valeurCompte(no) ;
    else
        throw new CompteInvalideException() ;
}
```

Exceptions en Java

Définir ses propres exceptions

- Pour créer une exception personnalisée :
 - Hériter de **Exception** ou d'une de ses sous-classes.
 - Ne pas hériter de **RuntimeException** ni de **Error**, car ces exceptions ne sont pas vérifiées.
 - Respecter la convention : inclure le mot **Exception** dans le nom de la classe.
 - Utilisation classique : lever l'exception avec **throw** et la gérer dans un bloc **try/catch/finally**.

Exceptions en Java

Un exemple de throws, throw, try et catch

```
public class CompteInvalideException extends Exception {  
    public CompteInvalideException() {  
        super("Numéro de compte invalide");  
    }  
    public CompteInvalideException(String message) {  
        super(message);  
    }  
}
```


Exceptions en Java

Un exemple de throws, throw, try et catch

```
public class NegativeValueException extends Exception {  
    public NegativeValueException() {  
        super("Valeur négative interdite");  
    }  
    public NegativeValueException(String message) {  
        super(message);  
    }  
}
```

Exceptions en Java

Un exemple de throws, throw, try et catch

```
public class Banque {  
    // Méthode qui peut lancer deux exceptions vérifiées  
    public static int compte(int no)  
        throws CompteInvalideException, NegativeValueException {  
        if (no < 0) {  
            throw new NegativeValueException("Le numéro de compte ne peut pas être  
                                                négatif");  
        } else if (no == 0) {  
            throw new CompteInvalideException();  
        }  
        return 100;  
    }  
}
```

Exceptions en Java

Un exemple de throws, throw, try et catch

```
public static void main(String[] args) {  
    int[] numeros = {5, 0, -3};  
    for (int no : numeros) {  
        try {  
            int solde = compte(no);  
            System.out.println("Solde du compte " + no + " : " + solde);  
        } catch (CompteInvalideException e) {  
            System.out.println("Erreur pour le compte " + no + " : " +  
e.getMessage());  
        } catch (NegativeValueException e) {  
            System.out.println("Erreur pour le compte " + no + " : " +  
e.getMessage());  
        }  
        System.out.println("Programme terminé normalement.");  
    }  
}
```

Exceptions en Java

Quiz

1. Les blocs try doivent avoir au moins un bloc catch
2. Les blocs try doivent avoir un bloc finally
3. Le bloc finally est obligatoire
4. Lors d'une exception dans un try
 - Le reste du bloc try est sauté
 - Le catch adéquat est exécuté, s'il est présent
 - Le catch adéquat est sauté si finally est présent
 - Le finally est exécuté, s'il est présent
5. Lorsqu'un try ne déclenche pas d'exception
 - Le finally est sauté, s'il est présent

Programmation Orientée objet Java

Pr. Ilyass OUAZZANI TAYBI

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence SI S5

Année universitaire : 2025/2026